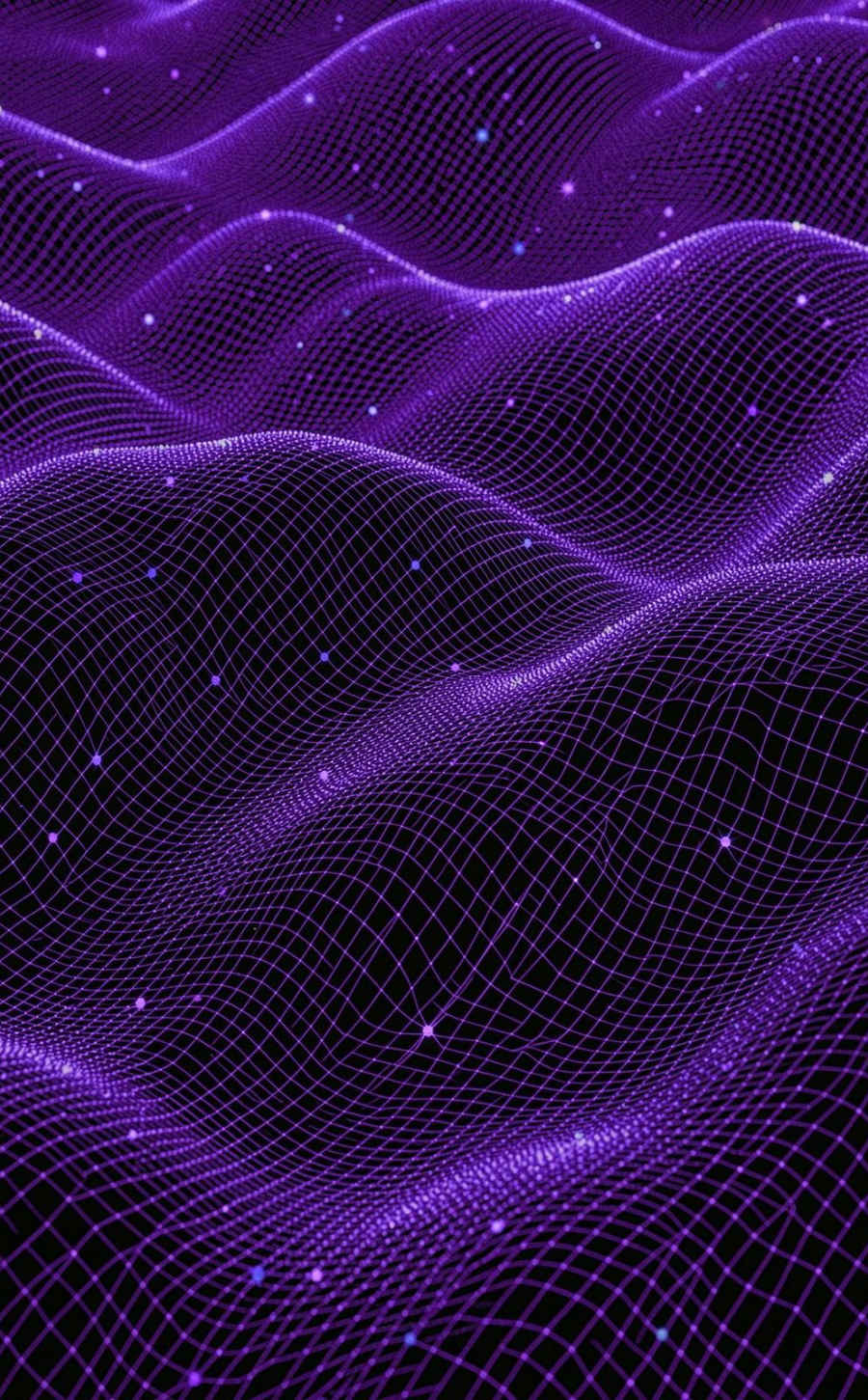




AI Research Scientist V2

A Multi-Agent AI System for Automated Research Workflows

Domain

AI-powered end-to-end research automation

Stack

Python 3.12 + OpenAI SDK + GitHub Models API

Architecture

6 specialized agents with shared state orchestration

Author

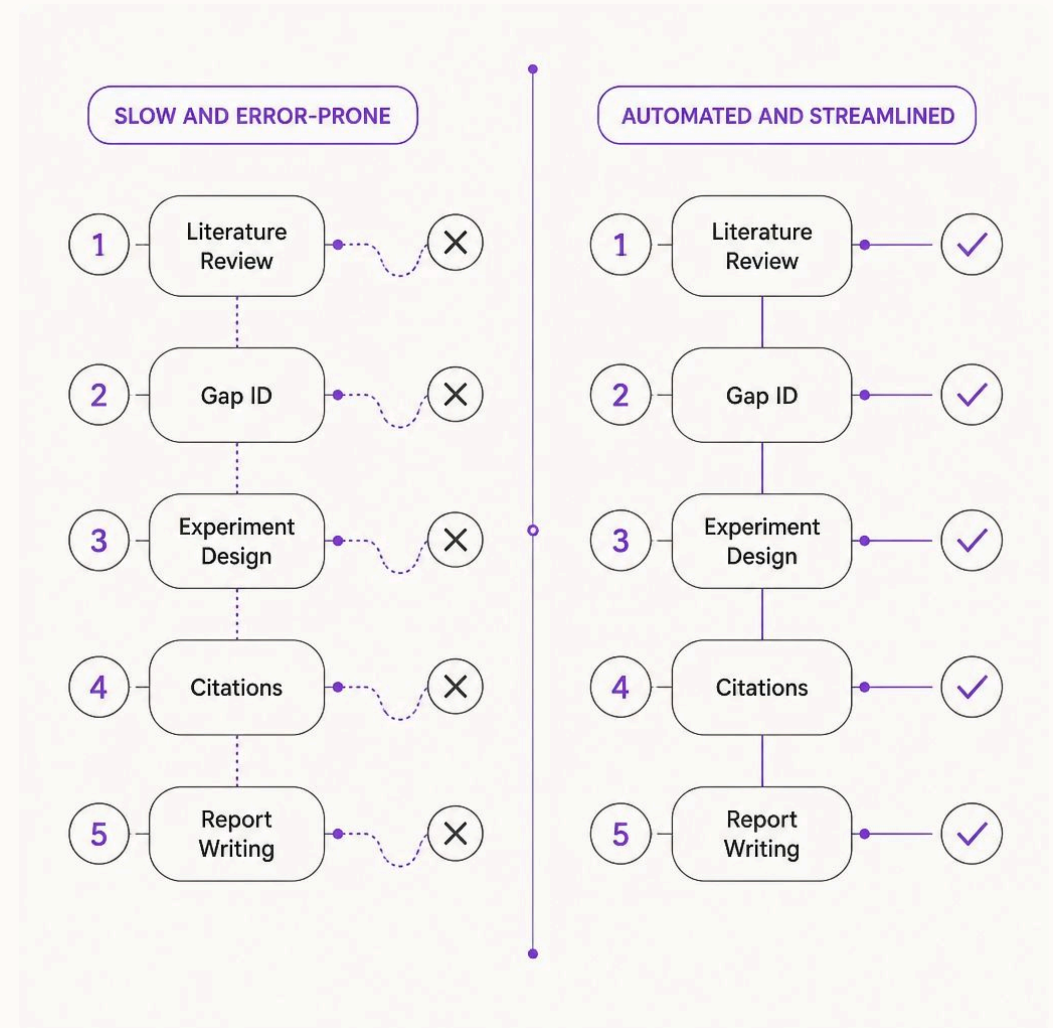
Shreya Borse

The Research Bottleneck

Traditional research workflows are slow, error-prone, and demand deep domain expertise at every stage. The cost is measured in hours — sometimes weeks.

- **Literature review:** Hours of reading, summarizing, and cross-referencing papers
- **Gap identification:** Requires deep domain expertise to spot what's missing
- **Experiment design:** Methodology, dataset curation, metric definition — all manual
- **Citation generation:** Error-prone, time-consuming, format-inconsistent
- **Report writing:** Synthesizing all stages into a coherent scientific document

"How can AI agents collaboratively automate the complete research lifecycle while keeping humans in the loop?"

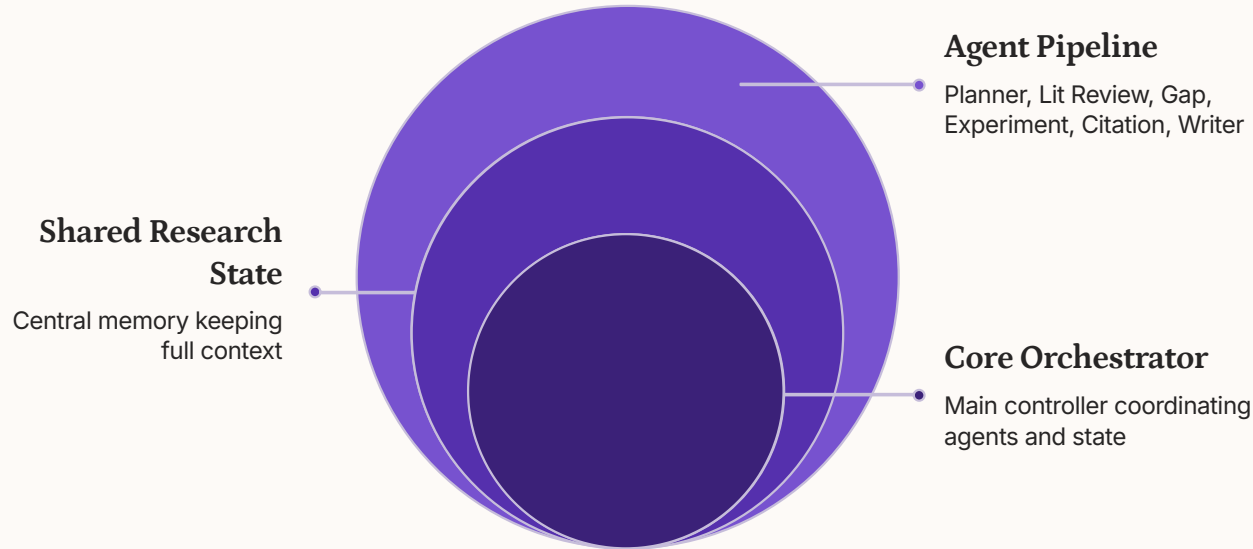


Solution: AI Research Scientist V2

A multi-agent AI system that automates the full research lifecycle through 6 specialized agents working in coordinated sequence.

1	Research Planning Structured roadmap from a single topic input
2	Literature Analysis Automated summarization of existing research
3	Gap Identification Systematic detection of open problems
4	Experiment Design Methodology, dataset, and metric recommendations
5	Citation Generation Structured reference list production
6	Report Generation Full markdown research report output

Multi-Agent Architecture



Architecture Highlights

- **Entry point:** Terminal Interface → Main Controller / Orchestrator
- **Shared Research State:** Central memory object maintaining full context across all agents
- **Agent pipeline:** Planner → Literature Review → Research Gap → Experiment Design → Citation → Scientific Writer
- **GitHub Models API:** Single LLM backend powering all agents via OpenAI-compatible SDK
- **Human Approval Gate:** Mandatory review checkpoint before final report is committed

The 6 Specialized Agents



Planner Agent

Decomposes topic into a prioritized research roadmap



Research Gap Agent

Surfaces limitations and unexplored opportunities



Citation Agent

Generates structured references in academic formats



Literature Review Agent

Summarizes key findings and methodologies



Experiment Design Agent

Proposes methodology, datasets, and metrics

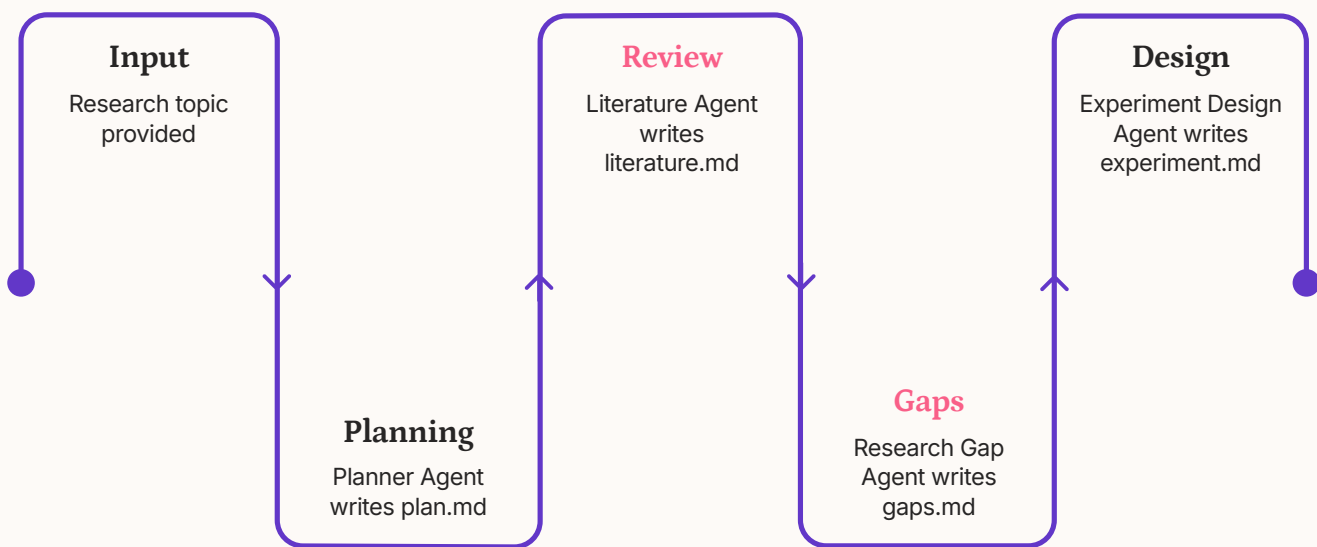


Scientific Writer Agent

Synthesizes all outputs into a publication-ready report

Agent Workflow & Handoff Mechanism

Agents execute sequentially, each reading from and writing to the **Shared Research State** for full context continuity.



01

Research Topic Input

User provides topic via terminal

02

Planner → Literature → Gaps

Sequential agents build on each prior output

03

Experiment → Citations → Report

Final agents synthesize all prior findings

04

Human Approval Gate

Reviewer validates before report is finalized

Technology Stack

1

Agent Modules + State Manager + Report Exporter

Internal components powering the multi-agent pipeline

2

Rich + python-dotenv

CLI rendering, progress indicators, secure API key management

3

OpenAI Python SDK + GitHub Models API

OpenAI-compatible endpoint powering all 6 agents

4

Python 3.12 Runtime

Modular, type-annotated agent architecture foundation

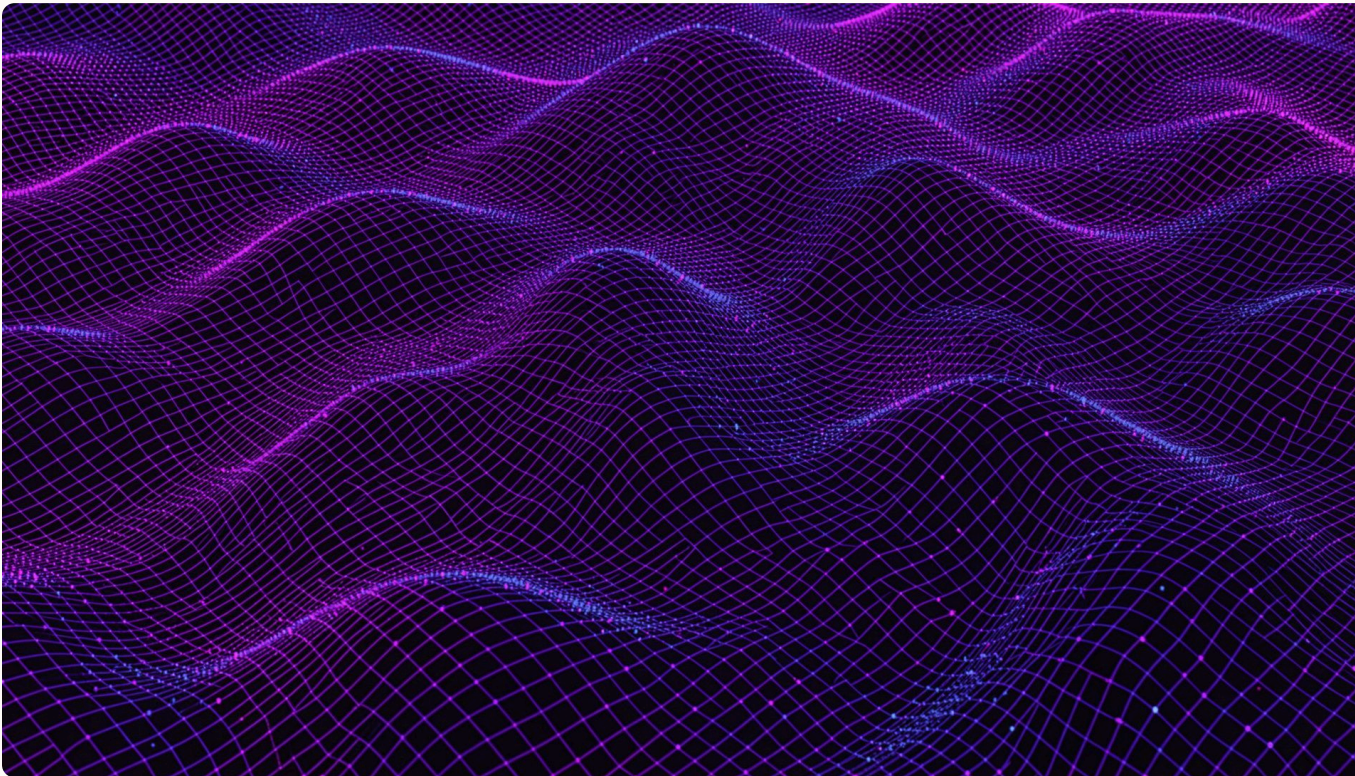
Implementation Details

How It Works Under the Hood

- **Agent execution:** Each agent accesses shared state, performs a specialized LLM call, and writes output back before passing control
- **Shared memory schema:** Stores topic, plan, literature findings, gaps, experiments, citations, and final report
- **Structured outputs:** Each agent exports a dedicated markdown file for traceability
- **Error handling:** Per-agent exception catching with graceful fallback via Rich console

Agent	Output File
Planner Agent	plan.md
Literature Review Agent	literature.md
Research Gap Agent	gaps.md
Experiment Design Agent	experiment.md
Citation Agent	citations.md
Scientific Writer Agent	final_report.md

Human-in-the-Loop & Advanced Features



AI-generated research passes through a mandatory human review gate before the final report is committed.

Planned Enhancements

- RAG integration for real-time academic paper retrieval
- Semantic Scholar / arXiv API connections
- Persistent research history · Web interface

✓ Planning & Reasoning

Structured topic decomposition before generation

✓ Shared Memory

Full context preserved across all 6 agents

✓ Modular Architecture

Each agent independently testable and replaceable

✓ Structured Outputs

Deterministic markdown file per agent stage

Example: LLMs in Healthcare

▶ LIVE WALKTHROUGH

Input: "Applications of Large Language Models in Healthcare"

Terminal Execution Log

```
$ python main.py  
> Topic: LLMs in Healthcare  
  
[1/6] Planner Agent... ✓ plan.md  
[2/6] Literature Agent... ✓ literature.md  
[3/6] Gap Agent... ✓ gaps.md  
[4/6] Experiment Agent... ✓ experiment.md  
[5/6] Citation Agent... ✓ citations.md  
[6/6] Writer Agent... ✓ final_report.md
```


Conclusion & Future Scope

"Transforming research from a manual process into an intelligent AI-assisted workflow."

Autonomous Collaboration

6 agents coordinate without human intervention between stages

Full Lifecycle Coverage

Topic input to publication-ready report in one pipeline run

Context-Aware Handoffs

Shared state ensures no information loss between agents

Human-Controlled

Approval gate keeps researchers in authority over final outputs

Future Roadmap

- 1

Academic Search APIs

Semantic Scholar + arXiv real-time retrieval
- 2

RAG Integration

Grounded, citation-verified generation
- 3

Multi-Modal Analysis

Images, tables, and figures in research pipeline
- 4

Cloud Deployment

Scalable, accessible research automation platform